Examen de l'UE "Architectures Avancées du Web" (HAI806I)

1e session 2025-26 - Michel Meynard - Durée : 1h - Barême : 10 points
UNIQUE document autorisé : une page A4 recto-verso manuscrite
16 avril 2026

Rédigez sur CE SUJET d'examen puis insérez-le dans la chemise où vous aurez collé vos étiquettes. Pensez à recopier votre numéro d'anonymat dans le cadre à droite.

Numéro d'anonymat

1 Gestion d'un panier d'articles

On souhaite réaliser à l'aide du framework Symfony v6, une application permettant à un utilisateur de stocker des articles en une certain quantité dans un panier (caddy) stocké en session :

- la page d'accueil contiendra une liste d'articles associés chacun à un champ de saisie de la quantité souhaitée
- à la soumission du formulaire, l'utilisateur soumettra comme nom de paramètre la référence de l'article choisi et comme valeur de ce paramètre la quantité choisie pour cet article
- la soumission du formulaire se fera en GET (et non en POST) afin de pouvoir visualiser la "query string" dans l'URL et on pourra soumettre plusieurs couples ref-quantité en même temps !
- un exemple : après soumission d'un formulaire d'achat de 3 téléphones Xiaomi (ref. 7) et de 1 portable Lenovo (ref. 1), la "query string" obtenue sera : `?7=3&1=1`
- au fur et à mesure des soumissions, le panier devra sauvegarder les différents articles choisis en écrasant éventuellement les quantités précédentes pour une même référence d'article
- vous devrez **ne pas développer** le formulaire mais développer la page permettant d'afficher et de gérer (2 boutons incrémenter/décrémenter pour chaque article) le panier courant comme dans la figure suivante :

Hard Discounter Votre magasin en ligne

		Quantité	Nom	Réf.	Prix U.	Prix Total
+	-	3	XIAOMI Redmi Note 8T 128 Go Bleu	7	252.00€	756€
+	-	1	PC portable Lenovo	1	499.99€	499.99€
						1255.99€

1.1 Modèle métier

On suppose qu'un projet Symfony v6 `SiteMarchand` a été créé avec tous les bundles nécessaires (maker, twig, ...). On suppose le modèle métier créé sous la forme **simplifiée** suivante :

```
// unique entité (en BD)
Article(id, ref UNIQUE, nom, prix, description)
ex. : (27, 7, "Xiaomi Redmi Note", 252, "téléphone ...") // produits
// classe PHP dont l'instance est stockée en SESSION
Panier(array artQte[ref => [quantite, article]])
ex. : [7 => [3, (27, 7, "Xiaomi Redmi Note", 252, "téléphone ...")],
       1 => [1, ( ...)]
      ]
// 3 téléphones et 1 portable
```

Question 1 (1 pt) Quelle commande console permet de lancer un assistant de création du controller `EditPanierController` ?

Question 2 (7 pts) Ecrire le code du contrôleur de la classe `EditPanierController` de route `/editPanier` qui :

Question 3 (2 pts) Ecrire le Twig de visualisation panier `editPanier.html.twig` en se concentrant sur le tableau privé de ses 2 premières colonnes (+ et -) : table de 5 colonnes !

— vérifie l'existence en session d'une variable "panier" sinon l'initialise (on suppose la classe Panier existante)
— vérifie la validité des paramètres de la "query string" (ref et qte de type entier différent de 0, ref correspond à une entité Article)
— le service Request possède une propriété query (paramètres GET) qui possède une méthode `all()` fournissant un tableau
— écrase (ou crée) dans la variable de session `panier` les lignes du tableau artQte qui ont une référence présente dans la "query string" en y associant un tableau contenant la quantité commandée ET l'article (entité)
— en cas d'erreur, redirige vers la route "erreur"
— affiche le rendu visuel situé dans `editPanier.html.twig`

Examen de Technologies avancées du web HAI806I
Lundi 4 mai 2026 - 13h-15h - SC 36.4
Une page recto-verso MANUSCRITE autorisée pour les deux parties de l'examen
-
Partie Pierre Pompidor

Pour toutes les questions, nous sommes dans le contexte de l'application d'ecommerce vue en TD.

Pour chaque document de la collection *articles* a été ajouté un attribut *stock* dont la valeur est le nombre de l'article concerné en stock. Si le stock est supérieur ou égal à 100 une remise de 15% sera proposée par le site.

Backend Angular 21 en SSR + Node.js + Express + MongoDB (4 points)

Donnez le code de l'**endpoint** qui dans le fichier *src/server.ts* reçoit une requête avec en paramètres une catégorie, une marque et un prix maximal, (chacun de ces paramètres peut avoir comme valeur * pour que respectivement, la catégorie, la marque ou le prix maximal soit quelconque), et renvoie le JSON des articles concernés.

Dans les données qui sont renvoyées, l'attribut stock a été supprimé (cette donnée étant confidentielle) et a été remplacé par un attribut *remise* de valeur *15* pour les articles ayant un stock supérieur ou égal à 100.

Frontend Angular 21 (6 points)

Composant *Products* (4 points)

Donnez les codes complets du composant *Products* (dossier *src/app/products*) qui a pour but de lister les articles sélectionnés (les fichiers de test et le CSS ne sont pas à écrire).

Ce composant utilise un service (classe *ProductService* du fichier *src/app/shared/products.service.ts*) qui conserve les références, noms, marques, prix et quantité des articles (vous n'avez pas à donner le code de ce service). Les données doivent être recopiées du service dans un signal propre à la classe du composant *Products*.

Le composant *Products* doit :

- afficher les articles sélectionnés (avec si elle existe, la remise de 15%) ;
- afficher un bouton qui permet d'ajouter un exemplaire de l'article dans le panier (le composant *Panier n'est pas à écrire*).

Template du composant de plus haut niveau (2 points)

Donnez le code du template du composant de plus haut niveau (*src/app/app.html*) qui fait apparaître :

- le formulaire permettant de rechercher les articles suivant les trois critères de recherche sur la catégorie, la marque et un prix maximal ;
- deux boutons permettant de demander au contrôleur de router soit le composant qui affiche les articles recherchés, soit celui qui affiche le panier ;
- la vue du composant routé.